

MEMORANDUM

TO: Alameda County Bar Association MCLE Attendees
FROM: John A. Kelly
DATE: September 9, 2026
RE: Take-Home Prompts for Grounding AI in the Law

This handout collects the electronic resources and the tested prompts from today's program, Beyond the Chatbot: Grounding AI in the Law. Each prompt is written so that you can copy it, adapt the bracketed parts to your matter, and use it the same day. The prompts assume you have first put the source material in front of the model, which is the discipline the program is about.

ELECTRONIC RESOURCES

Two repositories accompany this program. The first contains the complete text of all twenty-nine California codes in machine-readable form, synced nightly with the Legislature's own bulk data; each code is one plain-text file in the `codes` folder, so the Probate Code, for example, is the file named `CA Code - Probate Code.txt`. The second contains all fifty-eight county local rules in a structure suited to AI retrieval, in both full and civil-only extracts; where a court publishes its rules by title, the titles are separate files, so Alameda's probate rules are `Alameda_County_Title_7_Probate.txt` in the `rules` folder.

The California codes repository is at:

```
https://github.com/johnakelly-yahoo-com/california-codes
```

The county local rules repository is at:

```
https://github.com/johnakelly-yahoo-com/california-local-rules
```

HOW TO USE THESE PROMPTS

The prompt is only half of the work. A model can check or ground its answer only against material you actually give it, so every prompt below assumes you have placed the source in front of the model first: the statute text, the opinions, the treatise pages, the local rule, or your own document. How you do that depends on the tool you use.

If you use a general chatbot such as ChatGPT, Claude, Gemini, or Microsoft Copilot, start a new chat and find the attach or upload control, usually a paperclip or a plus sign. Add your source files, wait for them to finish uploading, and then paste the prompt. The model answers from what you attached. If you attach nothing, the model is working from memory and will invent things, which

is the failure mode this program is about.

If you use an agentic or coding tool such as Cursor, OpenAI Codex, or Claude CoWork, you can do more. These tools read whole folders of files and run programs. Put your sources in one folder, open that folder in the tool, and tell it to use those files. For the linter in Section 2, these tools can write the linter program and run it for you, then report which citations were found in the source and which were not. This is the harness version: grounding plus a real, repeatable check rather than a single chat answer.

If you use a dedicated legal platform such as Harvey, Legora, or Spellbook, the grounding is built in. In Harvey or Legora, load your documents into the matter, project, or vault first, then run the prompt so the platform answers only from those documents. In Spellbook, which works inside the Word document you have open, run the prompt against the draft together with the precedent or source language you have loaded. The rule never changes: load the source, then prompt against it.

1. CITATION CHECKING

The first task is confirming that an authority actually exists, and the second is putting it in the correct California form. The first runs on Westlaw, and the second runs in any chatbot.

Confirming that an authority exists has two steps. You paste a build prompt into a chatbot together with the document whose citations you need to check, and the model extracts the citations and builds the Westlaw search for you. The cases prompt and the statutes prompt are kept separate, but both searches can be run in one Westlaw session.

The first prompt handles cases.

```
When cases are cited in a letter, memo, or brief and need to be
confirmed, extract and build a Westlaw search for them.
Step 1. Extract the cases:
- List every case cited in the document with its full citation.
- Include only cases, not statutes, codes, or rules.
- Note the proposition each case is cited for.
Step 2. Build the Westlaw CI() search:
- Use the CI() (citation field) operator with the official reporter
  citation. Format: CI("[volume] [reporter] [page]")
- Join multiple citations with OR.
- Present the search as a single unwrapped line; Westlaw truncates
  multi-line pastes.
- Do not use cite(); it pulls cases that cite those cases and inflates
  the results.
- Do not include the case name; the reporter citation is sufficient.
- Database matters: CI() searches only the selected database. If the
  search mixes California and federal cases, run them separately
  (California in a CA database, federal in All Federal) or search in
  All Cases.
```

```
Example output (one unwrapped line):  
CI("7 Cal.4th 1238") OR CI("116 Cal.App.3d 311") OR CI("27 Cal.3d 167")
```

The second prompt handles statutes and works the same way.

```
When statutes are cited in a letter, memo, or brief and need to be  
confirmed, extract and build a Westlaw search for them.
```

```
Step 1. Extract the statutes:
```

- List every statute cited with its code name and section number.
- Include Code of Civil Procedure, Evidence Code, Probate Code, and the like.
- Note the proposition each statute is cited for.
- Do not include constitutional provisions; these are pulled separately.

```
Step 2. Build the Westlaw statute search:
```

- Use semicolon-separated Westlaw statute citations.
Format: [CODE ABBREVIATION] § [section number]
- Westlaw abbreviations: CA CIV PRO (Code of Civil Procedure),
CA EVID (Evidence Code), CA PROB (Probate Code),
CA BUS & PROF (Business and Professions Code).
- Present as a single unwrapped line.
- Run in the CA-ST-ANN database (California Statutes, Annotated).
- Westlaw may return extra results for neighboring or repealed sections; verify the count and discard sections not cited.

```
Example output (one unwrapped line):
```

```
CA CIV PRO § 2017.020; CA EVID § 954; CA PROB § 16060
```

The second step is running those searches in Westlaw and confirming what comes back. The workflow below downloads the authorities and verifies that each one matches the document.

```
After the model builds the searches, run them in Westlaw.
```

```
Step 3. Download (cases and statutes):
```

1. Paste each search into the Westlaw search bar; run cases in one search and statutes in another.
2. Verify the result count matches the number of authorities. For statutes the count may be slightly higher because Westlaw pulls neighboring or repealed sections; discard the extras.
3. Select All.
4. Download as PDF, single-column (not Original Image, which produces two-column scanned reporter pages that are hard to parse).

```
Step 4. Verify the citations:
```

- Cases: confirm each citation in the document matches the official reporter citation in the downloaded opinion, and that pinpoint pages support the proposition cited.
- Statutes: confirm each section number matches and the statutory text supports the proposition cited; discard any repealed or neighboring sections that are not cited.
- Conform every citation to the California Style Manual, including the year-before-case parenthetical and the bracketed parallel cite, using the next prompt.

```
Citation format reference:
```

- California Supreme Court: [vol] Cal.[series] [page]
- California Court of Appeal: [vol] Cal.App.[series] [page]
- Federal: [vol] F.[series] [page] or [vol] U.S. [page]
- Parallel cite, if that is all you have: [vol] Cal.Rptr.[series] [page]

To put citations in the correct California form, paste your text after this prompt in any chatbot. It needs no attachment, because it is a formatting task rather than a research task.

Make this text accurate pursuant to the California Style Manual, Fourth Edition (Jessen). Put the year before the case citation. Bracket the parallel cite. Do not include years for code section citations or for Rules of Court citations. Use these as the pattern:

Case:

(People v. Barton (1995) 12 Cal.4th 186
[47 Cal.Rptr.2d 569, 906 P.2d 531].)

Code:

(Code Civ. Proc. § 1005(b).)

Rule:

(Cal. Rules of Ct., Rule 3.1300(b).)

Rutter:

(Weil & Brown, Cal. Practice Guide: Civil Procedure Before Trial
(The Rutter Group 1997) ¶ 10:106 et seq., p. 10-43 et seq.)

2. CASE-LAW VERIFICATION: CLOSED CORPUS AND A LINTER

Use this approach when you cannot bulk-download the case law. You first attach or load a treatise or your confirmed opinions as the source, and you then verify the output mechanically rather than by eye.

To close the corpus, paste this prompt with the source attached or loaded.

I am giving you [the relevant treatise: Rutter, Witkin] as a closed corpus. Answer my question and cite only authorities that appear in this source. Anything not in this source is out of bounds. If the source does not answer the question, say so. Do not supply citations from memory.

To verify the result, have the model build a deterministic linter. This works best in Cursor, Codex, or Claude CoWork, which can write and run the program for you.

Write a deterministic linter I can run to check the output against the source. It is a program, not a judgment call. It must:

- take two inputs: the draft output and the source corpus text;
- extract every citation (and every quoted passage) from the draft;
- normalize whitespace and punctuation, then test by exact string match whether each one appears in the corpus;
- print each item as FOUND or NOT IN SOURCE, and exit nonzero if any item is NOT IN SOURCE;
- be reproducible: the same inputs always produce the same result.

Treat anything marked NOT IN SOURCE as a hallucination and flag it for my

```
review before substantive review. The linter reports; it does not edit.
```

For a further layer of assurance, you can wire a second AI model, called through an API, to evaluate the linter's output independently of the harness or chatbot that produced the draft. Because that reviewer is a separate system reading only the linter's FOUND and NOT IN SOURCE report, it does not inherit the first model's blind spots, and it can be set to run automatically on every draft rather than only when you ask. The result is a robust evaluation mechanism that sits outside the tool doing the drafting, which is the point of grounding the work in something a machine can check.

3. DOCUMENT-GROUNDED RESEARCH

The same discipline applies to the documents in your own file, where most real-world hallucination happens. You attach or load the document first, and then you paste the prompt.

```
Use only the attached [document, code section, or file] to answer. Quote the exact language you rely on, with its location (section, page, or line). If the attached material does not contain the answer, say "Not found in the provided source." Do not use outside knowledge or training data.
```

To check a draft against its sources, run the same deterministic linter from Section 2 against the draft and the source documents. Every statement, quotation, and citation in the draft must match language that actually appears in the sources, and anything marked not in source is flagged for your review.

4. STATUTORY RESEARCH GROUNDED IN THE PROBATE CODE

For a trusts and estates practice, the most useful single source in the codes repository is the Probate Code file. It is about 2.7 megabytes of plain text, small enough to attach to any of the chatbots above in one step, and it reflects the current text, including amendments that a model's training data cannot know. Download `CA Code - Probate Code.txt` from the codes repository, attach it, and paste this prompt with your question in the brackets.

```
Use only the attached Probate Code text to answer this question:
[state the question, for example: when a revocable trust becomes
irrevocable by the settlor's death, what notice must the trustee serve,
on whom, within what time, and what must the notice contain].
- Quote the exact statutory language you rely on, with the section
  number and subdivision, before you state any conclusion.
- If the answer turns on a defined term, quote the section that defines
  it.
- Note any cross-referenced section the attached text points to, and
  quote it if it bears on the answer.
- If the attached text does not answer the question, say "Not found in
  the provided source." Do not use outside knowledge or training data,
  and do not supply case law; cases are verified separately.
```

- Cite every section in California Style Manual form, following this pattern: (Prob. Code § 16061.7(a).)

The same prompt works against a single county's probate rules file from the local rules repository, and against the two together. Attach the Probate Code file and the county's probate title, and ask the model to answer from the statute first and then identify anything the local rule adds, such as a filing deadline, a required form, or a probate examiner practice. Because the county files are refreshed on the January and July rules cycles, check the repository's date stamp before relying on a local rule for a filing.

TO GO FURTHER

The two repositories above are part of the author's own grounding setup. You can download the file you need, whether a single code or one county's rules, and attach it as a source in any of the tools described above.

The written materials for this program, including the slide deck, this memorandum, the 58-county local rules survey used as the research case study, and a worked case study applying the same discipline to a single small matter, are collected at:

<https://blog.law-kelly.com/p/acba-presentation-materials.html>

For the ethics half of the program, the supporting documents (a model firm AI use policy, a court disclosure kit with a case management statement attachment, the county-by-county compliance matrix for AI local rules, and a memorandum on where the data goes in each way of working with AI) are collected at:

<https://blog.law-kelly.com/p/firm-ai-use-policy-package.html>

The full methodology, including the verification ladder, the treatise as a closed corpus, and the deterministic citation linter, is written up at:

<https://blog.law-kelly.com>